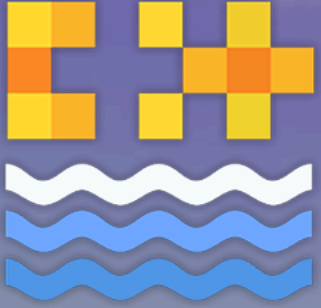




How To Keep Your Binaries Small?

Sandor DARGO

2024

Who am I?

Sándor DARGÓ

- Senior Engineer at Spotify
- Enthusiastic blogger writing sandordargo.com
- Passionate traveller
- Keen epicure
- Fortunate father of two



Why do we care about binary size?

- Devices with limited storage
- Devices with limited bandwidth
- Even environmental impact

Personal motivations

- Understand C++ binaries better
- Recognize patterns leading to bloat
- Understand the effects of modern practices

Agenda

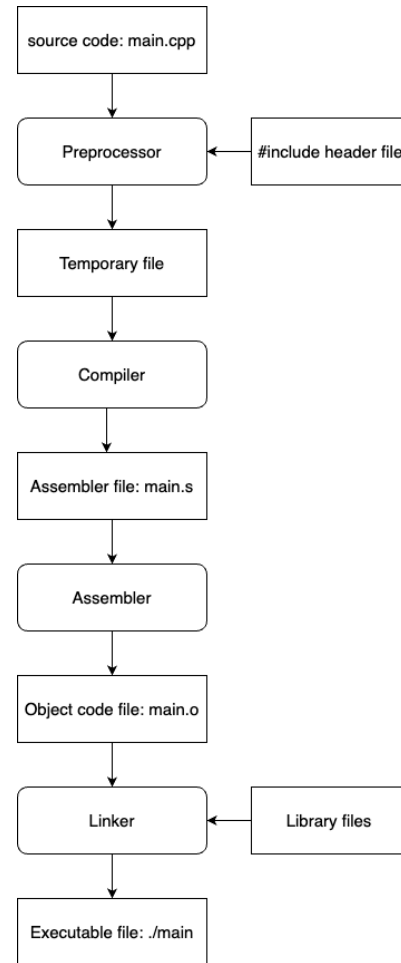
- Binary files 101
- Coding practices and binary size
- Compiler and linker settings
- Intersection of coding practices and compiler flags

Binary files 101

Mini-agenda

- How binaries are generated
- Main parts of an executable
- Their main characteristics
- Some different formats
- How to inspect binaries

The compilation process



Parts of an executable

- Header
- Text section
- Data sections (initialized vs uninitialized data)
- Read-only data section
- Symbol table
- Relocation information
- Import Table / Procedure Linkage Table
- Export table
- Exception handling information
- Debugging information

Main characteristics

- Single or cross platform?
- Linking/symbol resolution handling
- Position Independent Code (PIC)
- Endianness
- Architecture / extensibility

Most important formats

- a.out
- COFF
- ELF
- PE
- Mach-O

a.out

- assembler output
 - heritage of the 70s
 - not widely used anymore
- very simple:
 - header
 - text segment
 - data segment
 - symbol table
 - lacks support for
 - shared libraries
 - position independent code

Common Object File Format (COFF)

- Replacement for `a.out`
- First for *Unix*, later for *Windows*
- Supports large programs and debug information
- Has been replaced by other formats

Executable and Linkable Format (ELF)

Executable and Linkable Format (ELF)

- Used on *Unix*-like systems
- Supports
 - Shared libraries
 - Position Independent Code (PIC)
 - Debug information
- Designed for performance and extensibility

The ELF header

- 52 or 64 bytes large (32/64 bit)
- `e_ident` identifies (file type, encoding, versions)
- `e_type` - executable | shared object | object file
- `e_entry` VM address for program execution start
- `e_phoff`, `e_shoff`, offsets for program and section headers

What are some common sections?

- `.text` for executable code
- `.data` for initialized data
- `.bss` for uninitialized data
- `.rodata` for read-only data
- `.symtab` for symbol table
- `.strtab` for string table

Mach-O (Mach Object)

Mach-O (Mach Object)

- Designed for *macOS / iOS*
- Segments group related sections
- Supports
 - Shared libraries
 - Position Independent Code (PIC)
 - Debug information
- Designed for performance and extensibility

Header contains details, like

- CPU architecture
- File type
- Size and location of load commands

Load commands play a central role

- Segment Load Command
- Symbol Table Load Command
- Dynamic Symbol Table Load Command
- Dyld Info Load Command
- Code Signature Load Command
- Link Edit Load Command
- Version Load Command

`__TEXT` segment

- Read only segment containing executable code and constant data
- Usually the biggest segment
- Can be shared between different processes
- Contains several sections
 - `__text` section with the machine code
 - `__stub` and `__stub_helper` used for calling and referring external functions
 - `__const` section with all the constant data
 - `__cstring` with all the C-style, null-terminated strings
 - `__picsymbol_stub` containing PIC stubs

`__DATA` segment

- R/W area
- Must be copied by each process
- Most common sections
 - `__data` initialized vars with static storage duration
 - `__bss` similar but uninitialized ones
 - `__const` uninitialized extern globals
 - `__common` constant data needing to be relocated

Portable Executable format (PE)

Portable Executable format (PE)

- Supporting mainly windows
- Designed to be compact and efficient
- Supports
 - Shared libraries
 - Position Independent Code (PIC)
 - Debug information

COFF (Common Object File Format) header

- Signature `PE\0\0 / PE\0\0\0\0` 32-bit/64-bit
- Machine, target architecture (x86, x64)
- NumberOfSections
- TimeDataStamp
- PointerToSymbolTable: Points to the COFF symbol table for debugging information
- NumberOfSymbols: Number of entries in the COFF symbol table
- SizeOfOptionalHeader: Size of the optional header
- Characteristics: Flags indicating file attributes (DLL, executable, etc.)

Optional Header

- Magic:Formatoftheoptionalheader (`0x10B` for 32-bit, `0x20B` for 64-bit)
- MajorLinkerVersion and MinorLinkerVersion: Version of the linker used
- SizeOfCode: Size of executable code in bytes
All that you need to know about binary files 24
- AddressOfEntryPoint: Entry point of the program
- ImageBase: Preferred base address for loading into memory
- SectionAlignment and FileAlignment: Alignment of sections in memory and on disk
- MajorOperatingSystemVersion and MinorOperatingSystemVersion: Minimum OS version required
- Subsystem: Subsystem for program execution (e.g., Windows GUI, console)
- SizeOfImage: Size of the image in memory
- SizeOfHeaders: Size of all headers before the first section
- CheckSum: Checksum for the PE file
- SizeOfStackReserveandSizeOfStackCommit:Sizeofreserved and committed stack
- SizeOfHeapReserve and SizeOfHeapCommit: Size of reserved and committed heap
- NumberOfRvaAndSizes: Number of data-directory entries

No segments

- Sections specified in the section header
- `.rsrc` resource section
- Relocation section
- Debug info

Summary

- Each format tailored for different OSs
- Key characteristics and design goals are similar
- Implementations are vastly different

How to inspect binary files?

size for a quick overview

- Used in Unix-like OSs
- Gives size info about the different sections

size for a quick overview

- Used in Unix-like OSs
- Gives size info about the different sections

```
$ size global-array-10k-non-default-members  
__TEXT __DATA __OBJC others dec hex  
16384 81920 0 4294983680 4295081984 10001c000
```

`objdump` to see section sizes and more

- part of GNU binutils suite
- gets the file format, sections, sizes

objdump to see section sizes and more

- part of GNU binutils suite
- gets the file format, sections, sizes

```
$ objdump -h main_stdfunction
main_stdfunction:      file format mach-o arm64

Sections:
Idx Name              Size      VMA              Type
 0 __text             000068cc 0000000100000dc4 TEXT
 1 __stubs             00000168 0000000100007690 TEXT
 2 __gcc_except_tab    000001b0 00000001000077f8 DATA
 3 __const             0000012a 00000001000079a8 DATA
 4 __cstring           00000044 0000000100007ad2 DATA
 5 __unwind_info       000004e8 0000000100007b18 DATA
 6 __got               00000148 0000000100008000 DATA
 7 __const             00000228 0000000100008148 DATA
```

nm another utility of GNU binutils suite

- Lists symbols

nm another utility of GNU binutils suite

- Lists symbols

```
~$ nm -C std_function
s GCC_except_table1
U __Unwind_Resume
s vtable for std::__1::bad_function_call U vtable for std::length_error
U operator delete(void*)
U operator new(unsigned long)
t __clang_call_terminate
U __cxa_allocate_exception
U __cxa_begin_catch
```

Bloaty

- OS CLI tool developed by Google
- Provides insights into the size distribution of different sections within a binary
- Breaks down the binary into different sections and compares different parts
- Easy to combine different files

Bloaty

```
$ bloaty -s file std_function
```

FILE SIZE		VM SIZE		
-----		-----		
38.4%	15.3Ki	31.9%	15.3Ki	[__DATA_CONST]
29.2%	11.6Ki	24.4%	11.7Ki	[__TEXT]
11.4%	4.55Ki	9.5%	4.55Ki	String Table
5.2%	2.09Ki	4.4%	2.09Ki	__TEXT,__text
3.8%	1.50Ki	3.1%	1.50Ki	Symbol Table
3.4%	1.34Ki	2.6%	1.24Ki	[Mach-O Headers]
1.1%	464	0.9%	464	__DATA_CONST,__const
0.7%	294	0.6%	294	__TEXT,__const
0.6%	264	0.5%	264	__TEXT,__stubs
# . . .				
100.0%	39.9Ki	100.0%	48.0Ki	TOTAL

Ways of coding to reduce binary size

Mini-agenda

- Object initialization
- Member ordering
- Special member functions
- `virtual` functions
- Templates
- Passing functions around
- `constexpr` functions

**Not everything is a best practice that
shaves off a few bytes**

**Not everything is a best practice that
shaves off a few bytes**

Some things are simply trade-offs

Object initialization and binary sizes

Let's start with something small, yet big

```
#include <array>

struct Node {
    int a = 1, b = 1;
};

std::array <Node, 10'000> a;

int main() {}
```

- This piece of code generates an almost *100kB* binary

Let's look into it

```
clang++ -std=c++20 -O0 -stdlib=libc++ global-array-10k-non-default-members.cpp  
-o global-array-10k-non-default-members  
printf "%'d\n" $(wc -c < "global-array-10k-non-default-members")  
# 99,464
```

Let's look into it

```
$ bloaty -s file global-array-10k-non-default-members
```

FILE SIZE		VM SIZE		
-----		-----		
80.4%	78.1Ki	69.8%	78.1Ki	__DATA,__data
15.4%	14.9Ki	13.4%	15.0Ki	[__TEXT]
1.9%	1.88Ki	1.7%	1.88Ki	[__DATA]
1.0%	976	0.8%	872	[Mach-O Headers]
1.0%	952	0.8%	952	Code Signature
0.1%	120	13.4%	15.0Ki	[__LINKEDIT]
0.1%	88	0.1%	88	__TEXT,__unwind_info
0.0%	48	0.0%	48	Symbol Table
0.0%	32	0.0%	32	String Table
0.0%	20	0.0%	20	__TEXT,__text
0.0%	8	0.0%	8	Function Start Addresses
100.0%	97.1Ki	100.0%	112Ki	TOTAL

What contributes to this "huge" the size?

- The container
- The storage duration
- The initial value

Heap allocations to shrink your binary?

- Dynamic allocations might keep your binary smaller (but slows down your runtime)
- `std::list` and `std::vector` heap allocates
- C-style containers and `std::array` might be large

The previous example with alterations

Container	Binary size	Compile-time	Run-time
<code>std::array</code>	99K	2.8	0.39
C-style array	99K	0.75	0.39
<code>std::vector</code>	33-40K	2.8	0.75

Big *__DATA* segment for std::array

```
$ bloaty -s file global-array-10k-non-default-members
```

FILE SIZE		VM SIZE		
-----		-----		
80.4%	78.1Ki	69.8%	78.1Ki	__DATA,__data
15.4%	14.9Ki	13.4%	15.0Ki	[__TEXT]
1.9%	1.88Ki	1.7%	1.88Ki	[__DATA]
1.0%	976	0.8%	872	[Mach-O Headers]
1.0%	952	0.8%	952	Code Signature
0.1%	120	13.4%	15.0Ki	[__LINKEDIT]
0.1%	88	0.1%	88	__TEXT,__unwind_info
0.0%	48	0.0%	48	Symbol Table
0.0%	32	0.0%	32	String Table
0.0%	20	0.0%	20	__TEXT,__text
0.0%	8	0.0%	8	Function Start Addresses
100.0%	97.1Ki	100.0%	112Ki	TOTAL

Much smaller *__DATA* segment for std::vector

```
$ bloaty -s file global-vector-10k-non-default-members
```

FILE SIZE		VM SIZE		
-----		-----		
37.8%	15.8Ki	24.7%	15.8Ki	[__DATA_CONST]
22.2%	9.27Ki	14.6%	9.37Ki	[__TEXT]
14.9%	6.23Ki	9.7%	6.23Ki	String Table
11.0%	4.61Ki	7.2%	4.61Ki	__TEXT,__text
4.3%	1.80Ki	2.8%	1.80Ki	Symbol Table
3.4%	1.41Ki	2.1%	1.31Ki	[Mach-O Headers]
2.5%	1.06Ki	11.3%	7.24Ki	[__LINKEDIT]
1.2%	504	0.8%	504	Code Signature
0.9%	384	0.6%	384	__TEXT,__unwind_info
0.4%	192	0.3%	192	__TEXT,__stubs
0.4%	176	0.3%	176	__DATA_CONST,__got
0.4%	152	0.2%	152	Indirect Symbol Table
0.3%	136	0.2%	136	__TEXT,__gcc_except_tab
0.2%	96	0.1%	96	Function Start Addresses
0.0%	8	0.0%	8	__TEXT,__cstring
0.0%	4	0.0%	4	__TEXT,__init_offsets
0.0%	0	25.0%	16.0Ki	[__DATA]
0.0%	0	0.0%	24	__DATA,__common
100.0%	41.8Ki	100.0%	64.0Ki	TOTAL

***static* storage duration can increase the
binary size**

What are the different storage durations?

- automatic (non-static, non-extern, non-thread-local, local variables)

```
class MyClass {  
    public:  
    // ..  
    int foo(int bar /* automatic storage duration */) {  
        auto result = m_num * bar; // automatic  
        // ..  
        return result;  
    }  
    private:  
    int m_num; // automatic storage duration  
}
```

What are the different storage durations?

- automatic (non-static, non-extern, non-thread-local, local variables)
- static (static + namespace level + extern variables)

```
namespace foo {  
    int bar = 42; // static SD  
    extern int myvar; // static SD  
    void foo() {  
        static int baz = 66; // static SD  
        // ...  
    }  
}
```

What are the different storage durations?

- automatic (non-static, non-extern, non-thread-local, local variables)
- static (static + namespace level + extern variables)
- dynamic (heap allocated variables)

```
int* p = new int(7);  
throw 123;
```

What are the different storage durations?

- automatic (non-static, non-extern, non-thread-local, local variables)
- static (static + namespace level + extern variables)
- dynamic (heap allocated variables)
- thread-local (`thread_local` variables)

```
struct S {  
    thread_local static int tlm;  
};  
  
thread_local int S::tlm = 0; // "static" does not appear here
```

static storage duration can increase the binary size

- Global and `static` variables have static storage duration
- Data might end up in the `TEXT` or `DATA` segment depending on `const`ness and initialization
- Local containers won't make huge binaries, even if they are `const`
- `constexpr` locals can also take a lot of space!

Let's see some numbers!

<code>std::array</code>	Binary size	Compile-time	Run-time
global	99K	2.2	0.25
local	33K	2.2	0.87
local static	16-99K	2.2	0.89
local static const	16-82K	2.2	0.84
local constexpr	16-99K	2.8	0.93

Bloaty of a global array

```
$ bloaty -s file global-array-10k-non-default-members
```

FILE SIZE		VM SIZE		
-----	-----	-----	-----	
80.4%	78.1Ki	69.8%	78.1Ki	__DATA,__data
15.4%	14.9Ki	13.4%	15.0Ki	[__TEXT]
1.9%	1.88Ki	1.7%	1.88Ki	[__DATA]
1.0%	976	0.8%	872	[Mach-O Headers]
1.0%	952	0.8%	952	Code Signature
0.1%	120	13.4%	15.0Ki	[__LINKEDIT]
0.1%	88	0.1%	88	__TEXT,__unwind_info
0.0%	48	0.0%	48	Symbol Table
0.0%	32	0.0%	32	String Table
0.0%	20	0.0%	20	__TEXT,__text
0.0%	8	0.0%	8	Function Start Addresses
100.0%	97.1Ki	100.0%	112Ki	TOTAL

Bloaty of a local array

```
$ bloaty -s file local-array-10k-non-default-members
```

FILE SIZE		VM SIZE		
-----	-----	-----	-----	
48.7%	16.0Ki	33.3%	16.0Ki	[__DATA_CONST]
44.6%	14.6Ki	30.7%	14.7Ki	[__TEXT]
2.9%	976	1.8%	872	[Mach-O Headers]
1.3%	432	0.9%	432	Code Signature
0.9%	312	0.6%	312	__TEXT,__text
0.5%	160	0.3%	160	String Table
0.5%	160	31.9%	15.3Ki	[__LINKEDIT]
0.3%	112	0.2%	112	Symbol Table
0.3%	104	0.2%	104	__TEXT,__unwind_info
0.0%	8	0.0%	8	Function Start Addresses
0.0%	8	0.0%	8	Indirect Symbol Table
0.0%	8	0.0%	8	__DATA_CONST,__got
100.0%	32.9Ki	100.0%	48.0Ki	TOTAL

Bloaty of a local constexpr array (-O0)

```
$ bloaty -s file local-constexpr-array-10k-non-default-members
```

FILE SIZE		VM SIZE		
-----	-----	-----	-----	
80.3%	78.1Ki	69.8%	78.1Ki	__TEXT,__const
16.4%	16.0Ki	14.3%	16.0Ki	[__DATA_CONST]
1.1%	1.11Ki	0.9%	1.01Ki	[Mach-O Headers]
1.0%	960	0.8%	960	Code Signature
0.6%	584	0.6%	688	[__TEXT]
0.2%	168	13.3%	14.9Ki	[__LINKEDIT]
0.1%	100	0.1%	100	__TEXT,__text
0.1%	88	0.1%	88	__TEXT,__unwind_info
0.1%	64	0.1%	64	String Table
0.1%	64	0.1%	64	Symbol Table
0.0%	16	0.0%	16	Indirect Symbol Table
0.0%	16	0.0%	16	__DATA_CONST,__got
0.0%	12	0.0%	12	__TEXT,__stubs
0.0%	8	0.0%	8	Function Start Addresses
100.0%	97.2Ki	100.0%	112Ki	TOTAL

Bloaty of a local constexpr array (-O3)

```
$ bloaty -s file local-constexpr-array-10k-non-default-members
```

FILE SIZE		VM SIZE		
-----		-----		
91.6%	15.1Ki	47.5%	15.2Ki	[__TEXT]
4.9%	824	2.2%	720	[Mach-O Headers]
1.9%	320	1.0%	320	Code Signature
0.6%	104	48.8%	15.6Ki	[__LINKEDIT]
0.5%	88	0.3%	88	__TEXT,__unwind_info
0.2%	32	0.1%	32	String Table
0.2%	32	0.1%	32	Symbol Table
0.0%	8	0.0%	8	Function Start Addresses
0.0%	8	0.0%	8	__TEXT,__text
100.0%	16.5Ki	100.0%	32.0Ki	TOTAL

Initial values might matter

- Change the values in the previous example from 1 to 0

Initial values might matter

- Change the values in the previous example from 1 to 0

```
#include <array>

struct Node {
    int a = 0 /* 1 */, b = 0 /* 1 */;
};

std::array <Node, 10'000> a;

int main() {}
```

Initial values might matter

- Change the values in the previous example from 1 to 0

```
#include <array>

struct Node {
    int a = 0 /* 1 */, b = 0 /* 1 */;
};

std::array <Node, 10'000> a;

int main() {}
```

- And your binary will shrink from 99k to 16k

The compiler can optimize initialization

The compiler can optimize initialization

- Instead of a long section of initialization:

```
.section    __DATA,__data
    .globl  _a                                ; @a
    .p2align 2
_a:
    .long   1                                ; 0x1
    .long   1                                ; 0x1
    .long   1                                ; 0x1
    // ...
    .long   1                                ; 0x1
```

The compiler can optimize initialization

- You might only have one command

```
.zerofill __DATA,__common,_segtree,80000,2
```

What if we add another member?

What if we add another member?

- Add something that can be (zero-)initialized at compile-time and `.zerofill` will be still used

What if we add another member?

- Add something that can be (zero-)initialized at compile-time and only `.zerofill` will be still used

```
struct Node {  
    int a = 0, b = 0, c = 0;  
};
```

What if we add another member?

```
bloaty -s file global-array-10k-default-members
```

FILE SIZE		VM SIZE		
-----		-----		
90.6%	15.0Ki	9.4%	15.1Ki	[__TEXT]
5.8%	976	0.5%	872	[Mach-O Headers]
1.8%	304	0.2%	304	Code Signature
0.7%	120	9.8%	15.6Ki	[__LINKEDIT]
0.5%	88	0.1%	88	__TEXT,__unwind_info
0.3%	48	0.0%	48	Symbol Table
0.2%	32	0.0%	32	String Table
0.0%	8	0.0%	8	Function Start Addresses
0.0%	8	0.0%	8	__TEXT,__text
0.0%	0	6.8%	10.8Ki	[__DATA]
0.0%	0	73.2%	117Ki	__DATA,__common
100.0%	16.5Ki	100.0%	160Ki	TOTAL

What if we add another member?

- Add something that can be (zero-)initialized at compile-time and only `.zerofill` will be still used
- Add something that needs runtime initialization and the binary will grow as more code has to be generated

What if we add another member?

- Add something that can be (zero-)initialized at compile-time and only `.zerofill` will be still used
- Add something that needs runtime initialization and the binary will grow as more code has to be generated

```
struct Node {  
    int a = 0, b = 0;  
    std::string c;  
};  
/*  
.zerofill __DATA,__common,_a,320000,3  
.section __DATA,__mod_init_func,mod_init_funcs  
.p2align 3, 0x0  
.quad __GLOBAL__sub_I_global_array_10k_default_members.cpp  
.subsections_via_symbols  
*/
```

What if we add another member?

```
$ bloaty -s file global-array-10k-default-members
```

FILE SIZE		VM SIZE		
-----		-----		
48.6%	16.0Ki	4.3%	16.0Ki	[__DATA_CONST]
44.2%	14.5Ki	4.0%	14.6Ki	[__TEXT]
3.8%	1.26Ki	0.3%	1.16Ki	[Mach-O Headers]
1.3%	432	0.1%	432	Code Signature
0.5%	168	4.2%	15.3Ki	[__LINKEDIT]
0.4%	144	0.0%	144	String Table
0.3%	112	0.0%	112	Symbol Table
0.3%	112	0.0%	112	__TEXT,__text
0.3%	96	0.0%	96	__TEXT,__unwind_info
0.1%	24	0.0%	24	__TEXT,__stubs
0.0%	16	0.0%	16	Indirect Symbol Table
0.0%	16	0.0%	16	__DATA_CONST,__got
0.0%	8	0.0%	8	Function Start Addresses
0.0%	4	0.0%	4	__TEXT,__init_offsets
0.0%	0	2.0%	7.50Ki	[__DATA]
0.0%	0	84.9%	312Ki	__DATA,__common
100.0%	32.9Ki	100.0%	368Ki	TOTAL

Member ordering and binary sizes

Let's first talk about padding and alignment

- Padding is the extra bytes between members
- Alignment requirement is the for memory addresses to start at certain locations
 - 4 byte `int` to start at an address divisible by 4
 - 8 byte `double` to start at an address divisible by 8

Bad member order leads to higher object size

```
struct UnoptimalOrder {  
    int i1;        // 4 bytes  
    double d1;     // 8 bytes  
    int i2;        // 4 bytes  
};  
  
static_assert(sizeof(UnoptimalOrder) == 24);
```

Order them decreasing order to limit size

```
struct OptimalOrder {  
    double d1;    // 8 bytes  
    int i1;       // 4 bytes  
    int i2;       // 4 bytes  
};  
  
static_assert(sizeof(OptimalOrder) == 16);
```

This won't always reduce binary size

- The compiler can often optimize
- It matters when members are not initialized to their default value
- It's still a good practice for better cache friendliness and memory footprint

Special functions and binary sizes

Let's start with this piece of code

```
#include <array>
class C {
public:
    C() = default;
    ~C() = default;
    C(const C&) = default;
    C(C&&) = default;
    C& operator=(const C&) = default;
    C& operator=(C&&) = default;

    // ...
};
std::array<C, 10000> a{};
int main() {}
```

Let's have 6 versions of almost the same class

- inline `default` special functions
- inline empty special functions
- following the rule of zero (RoZ)
- each with either `virtual` or non-`virtual` destructors

The results

Version	Binary size
default non- <code>virtual</code>	16K
empty non- <code>virtual</code>	17-34K
RoZ non- <code>virtual</code>	16K
default <code>virtual</code>	116K / 198k
empty <code>virtual</code>	33-34K
RoZ <code>virtual</code>	116K / 198k

virtual matters, **default** != empty

- polymorphic classes always have an overhead
- **default** implementations differ from empty ones
- classes are rarely used as such...

Let's move them to their own file(s)!

- Move `main()` to another file
- Move definitions out of line
- Compare `virtual` and non-`virtual` results separately

Defaults in the header seems to have a big advantage!

Version	Binary size
default non- <code>virtual</code> header/only	16K
default non- <code>virtual</code> separated	34K
empty non- <code>virtual</code> header-only	15-34K
empty non- <code>virtual</code> separated	34K

This data is a bit like Newtonian physics

- It describes well what happens on small scale
- But it doesn't apply on big scale

`=default` or not to `=default` ?

- Follow the rule of zero
- Otherwise the rule of five
- And use `=default` whenever possible!

But where should we `=default` ?

- It makes sense at declaration time
 - It's readable for us
 - It's helpful for the compiler
- **Why would you do something else?**

What happens when you `=default` out-of-line?

- The compiler cannot inline the body of the `default`ed function
- The same piece of code won't be repeated over and over again
- On a big project scale, out-of-line `default`s shrink the executable

Small size != best practice

- Should we go for empty instead of `=default` implementation?
 - No!
- Should we `default` in the `.cpp` instead of the `.h`?
 - Maybe, it depends...

Where to `default`?

- In the header, by default
- In the `.cpp` to avoid inlining and to gain some space
 - It's still counterintuitive
 - And misleading
 - Document the reason
 - Otherwise, someone will undo it

Not declared move operations might hurt you

Let's have a look at the Hinnant-table

Special Members

compiler implicitly declares

user declares		default constructor	destructor	copy constructor	copy assignment	move constructor	move assignment
	Nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
	Any constructor	not declared	defaulted	defaulted	defaulted	defaulted	defaulted
	default constructor	user declared	defaulted	defaulted	defaulted	defaulted	defaulted
	destructor	defaulted	user declared	defaulted	defaulted	not declared	not declared
	copy constructor	not declared	defaulted	user declared	defaulted	not declared	not declared
	copy assignment	defaulted	defaulted	defaulted	user declared	not declared	not declared
	move constructor	not declared	defaulted	deleted	deleted	user declared	not declared
	move assignment	defaulted	defaulted	deleted	deleted	not declared	user declared

Not declared move operations might hurt you

- If only copy operations are available, copies will be made
- Even when a move would be more reasonable
- They incur more assembly, so the binary will be bigger
- Follow the rule of 5 if you cannot apply the rule of zero!

Virtuals and binary sizes

What does the `virtual` keyword do?

- Enables redefining methods in derived classes
- Defers function binding to runtime

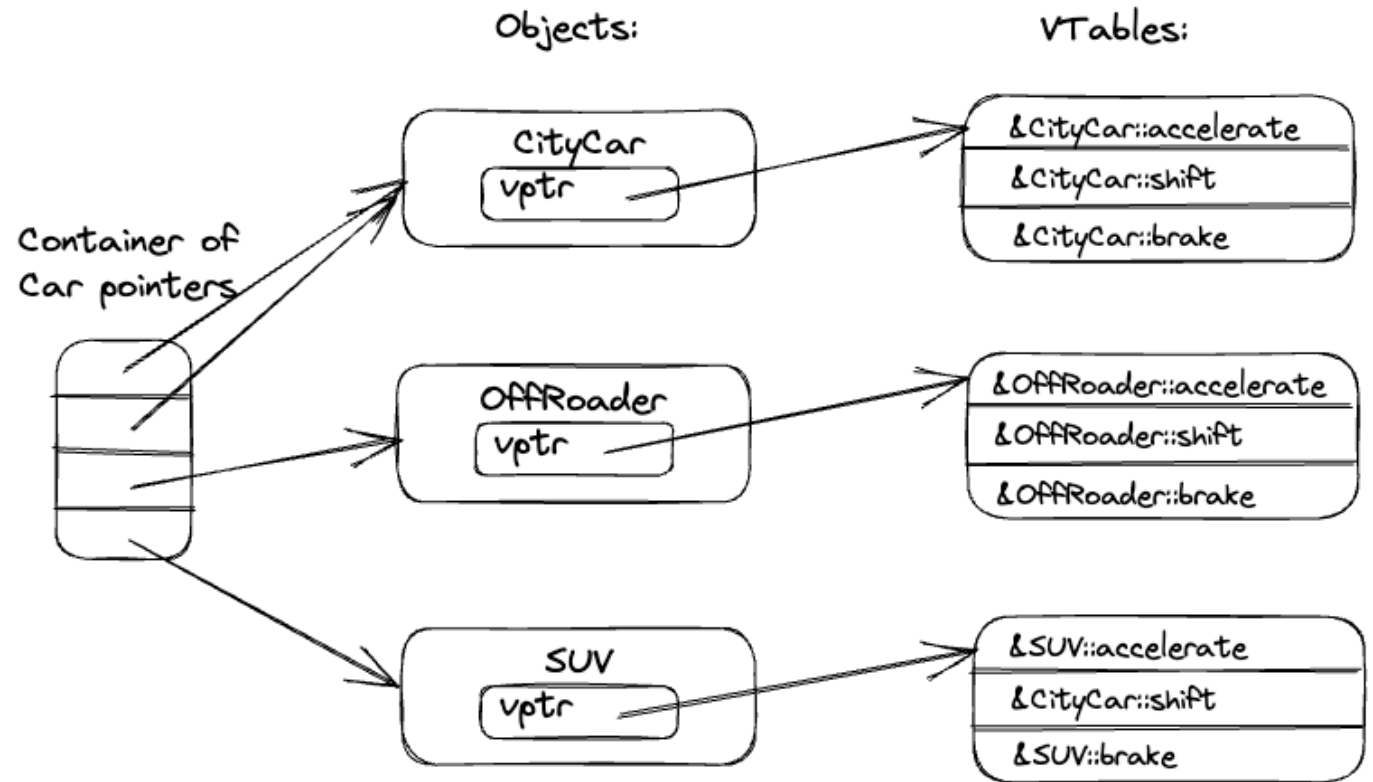
What does the `virtual` keyword do?

- Enables redefining methods in derived classes
- Defers function binding to runtime

```
class Car {  
public:  
    virtual void accelerate() = 0;  
    virtual void shift() = 0;  
    virtual void accelerate() = 0;  
};  
  
class CityCar: public Car { /* */ };  
class OffRoader: public Car { /* */ };  
class SUV: public CityCar { /* */ };
```

How does it look like?

- *vtable* for each class
- *vtable* contains pointers to each **virtual** function
- each object has a *vpointer*



Our hypothesis

- A *vtable* would increase the size of a binary significantly
- Additional `virtual` functions would further increase the size only by a little

Let's run some experiments!

```
class VirtualExperiment {  
public:  
    VirtualExperiment() = default;  
    VirtualExperiment(int a, int b, int c) : m_a(a), m_b(b), m_c(c) {}  
    /* virtual */ ~VirtualExperiment() = default;  
  
    /* virtual */ int getA() const { return m_a; }  
    /* virtual */ int getB() const { return m_b; }  
    /* virtual */ int getC() const { return m_c; }  
private:  
    int m_a = 0;  
    int m_b = 0;  
    int m_c = 0;  
};
```

The results

Version	Binary size
no <code>virtual</code>	16,879
only <code>virtual</code> dtor	281,505
one <code>virtual</code>	281,553
two <code>virtual</code> s	281,601
three <code>virtual</code> s	281,649

Different initial values to avoid optimization!

```
class VirtualExperiment {  
public:  
    VirtualExperiment() = default;  
    VirtualExperiment(int a, int b, int c) : m_a(a), m_b(b), m_c(c) {}  
    /* virtual */ ~VirtualExperiment() = default;  
  
    /* virtual */ int getA() const { return m_a; }  
    /* virtual */ int getB() const { return m_b; }  
    /* virtual */ int getC() const { return m_c; }  
private:  
    int m_a = 1;  
    int m_b = 1;  
    int m_c = 1;  
};
```

Only the non-polymorphic size changed

Version	Binary size
no virtual	148,975
only virtual dtor	281,505

What can we say?

- A single `virtual` destructor has a big price
- Even if the class is suboptimal for optimization
- Once there is one `virtual` function, the rest has no big price

Templates and binary size

Understand when you use templates

- Many say, oh we don't use templates
- In reality, almost all of us do, through the standard library
- But with modern C++, `auto` can also deceive the less experienced

Understand when you use templates

- Many say, oh we don't use templates
- In reality, almost all of us do, through the standard library
- But with modern C++, `auto` can also deceive the less experienced

```
auto add(auto a, auto b); // this is a template without the keyword
```

Use minimal templates

- Each template expansion copies code
- The longer the code, the more is copied
 - Even though the compiler might optimize
- So keep your template small (both functions and classes)
- And extract the rest

Use minimal function templates

- Instead of this:

```
template <typename T>
void foo(T t) {
    int r = t.bar();
    // long piece of code
    // not using T
    // ..
}
```

Use minimal function templates

- Use this:

```
void extracted(int r) {  
    // long piece of code  
    // not using T  
    // ..  
}  
  
template <typename T>  
void foo(T t) {  
    int r = t.bar();  
    extracted(r);  
}
```

Use minimal class templates

- Instead of this:

```
template <typename T>
struct Widget {
    int m_i;
    T m_j;

    void foo(int x) { /* no useage of T */}
    void bar(T y) { /* some usage of T */}
};
```

Use minimal class templates

- Use this:

```
struct WidgetBase {  
    int m_i;  
  
    void foo(int x) { /* no useage of T */ }  
};  
  
template <typename T>  
struct Widget : WidgetBase {  
    T m_j;  
  
    void bar(T y) { /* some usage of T */ }  
};
```

Extern templates

What do extern templates do?

- `extern` indicates that the definition should be found by the linker

What do extern templates do?

- `extern` indicates that the definition should be found by the linker

```
template <typename T>
class Wrapper {
public:
    // ...
private:
    T wrapped;
};

// No object code will be generated for Wrapper<int> in
// this translation unit
extern template class Wrapper<int>;
```

What do extern templates do?

- Limit the number of times code will be generated
- Good for compile time
- Good for binary size

How to use extern templates? Option 1)

- Put `extern template class Wrapper<int>` in every TU where you instantiate `Wrapper<int>`
- Straightforward
- A bit repetitive
- Easy to forget about it

How to use extern templates? Option 2)

- Put `extern template class Wrapper<int>` in the header (`wrapper.h`) where you declare `Wrapper<T>`
- As such everyone who includes `wrapper.h` will have the declaration
- In `wrapper.cpp` put the explicit template declaration
`template class Wrapper<int>;`

How to use extern templates? Option 2)

```
// wrapper.h
template <typename T>
class Wrapper {
public:
    explicit Wrapper(T t) : wrapped(t) {}
    T get() const { return wrapped; }
    void set(T t) { wrapped = t; }
private:
    T wrapped;
};

// No object code will be generated for Wrapper<int>
// in translation units including this file
extern template class Wrapper<int>;

// wrapper.cpp
#include "wrapper.h"

// explicit template instantiation
template class Wrapper<int>;
```

```
#include "wrapper.h"

#include <iostream>

// the below line is included from wrapper.h
// extern template class Wrapper<int>;

namespace moduleA {
    void foo() {
        Wrapper w{42};
        w.set(51);
        std::cout << w.get() << '\n';
    }
}
```

Are `extern` templates worth it?

- Limited gains within one library
 - Especially if new TUs have to be introduced
- More significant across libraries
 - If used with widely used types
- No inlining without IPO/LTO enabled
 - As if definitions would be in `.cpp` files

Avoid bloaty class templates

Avoid bloaty class templates

- The more functionalities provide a template the more bloaty it is
- If size is a critical concern for you, think twice before you introduce more
- `std` / `boost` etc. types are no exception

How to pass functions to functions?

We have plenty of options

- `std::function`

We have plenty of options

- `std::function`

```
void foo(std::function<bool(std::string)> fun, std::string bar) {  
    [[maybe_unused]] auto r = fun(bar);  
    // ...  
}
```

We have plenty of options

- `std::function`
- Bare templates

```
template<typename Callable>
void foo(Callable fun, std::string bar) {
    [[maybe_unused]] auto r = fun(bar);
    // ...
}
```

We have plenty of options

- `std::function`
- Not so bare templates

```
template<std::invocable<std::string> Callable>
requires // requires clause
    requires (Callable callable, std::string s) { // requires expression
        { callable(s) } -> std::same_as<bool>;
    }
void foo(Callable fun, std::string bar) {
    [[maybe_unused]] auto r = fun(bar);
    // ...
}
```

We have plenty of options

- `std::function`
- Not so bare templates
- Function pointers

We have plenty of options

- `std::function`
- Not so bare templates
- Function pointers

```
void foo(bool(* fun)(std::string), std::string bar) {  
    [[maybe_unused]] auto r = fun(bar);  
    // ...  
}
```

We have plenty of options

- `std::function`
- Not so bare templates
- Function pointers in a more readable way

```
using Function = bool (*)(std::string);  
void foo(Function fun, std::string bar) {  
    [[maybe_unused]] auto r = fun(bar);  
    // ...  
}
```

Different options, different advantages

- Function pointers have an obscure syntax
- `std::function` works out of the box with any kind of lambdas
- Templates are quite specializable with concepts

Let's run some experiments

```
int performOperation(int(* op)(int, int), int lhs, int rhs) { return op(lhs, rhs); }
int main() {
    auto add = [](int lhs, int rhs) {return lhs + rhs;};
    auto subtract = [](int lhs, int rhs) {return lhs - rhs;};
    auto multiply = [](int lhs, int rhs) {return lhs * rhs;};
    auto extra = [](int lhs, int rhs) {return 2*lhs * rhs;};
    auto multiplyBy2 = [](int lhs) {return 2 * lhs;};
    std::cout << performOperation(add, 42, 51) << '\n';
    std::cout << performOperation(subtract, 42, 51) << '\n';
    std::cout << performOperation(multiply, 42, 51) << '\n';
    std::cout << performOperation(extra, 42, 51) << '\n';
    std::cout << performOperation(multiplyBy2, 42) << '\n';
    return 0;
}
```

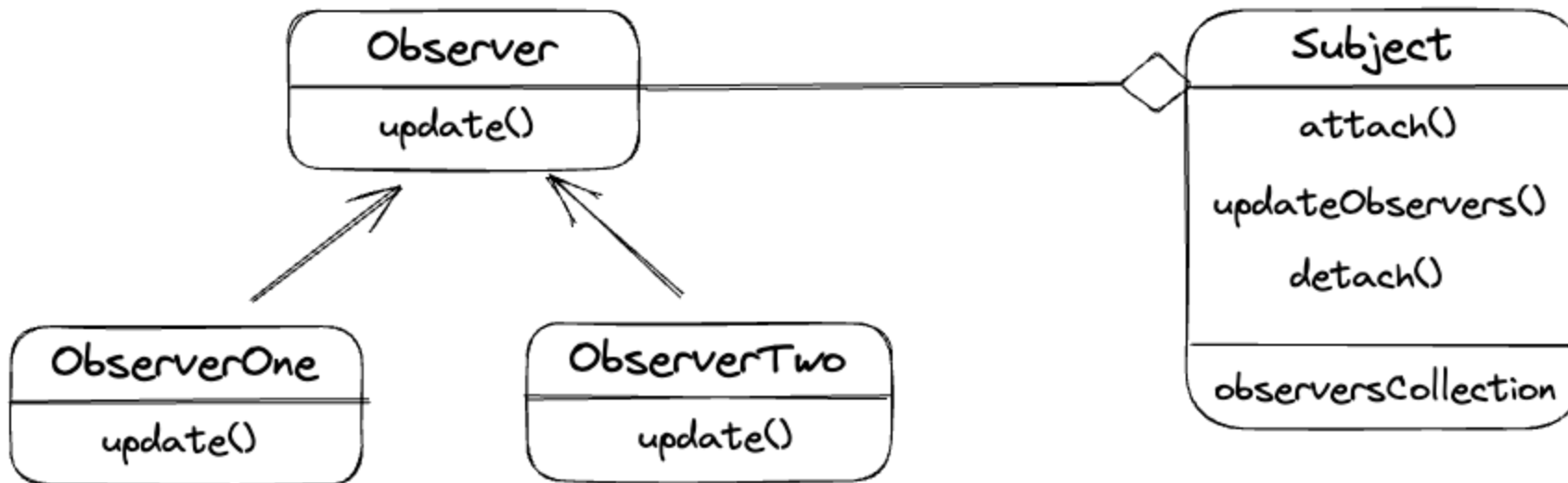
Flexibility has a price

Version	Binary size
<code>std::function</code> -O3	40,189
templates -O3	35,259
function pointer -O3	35,341

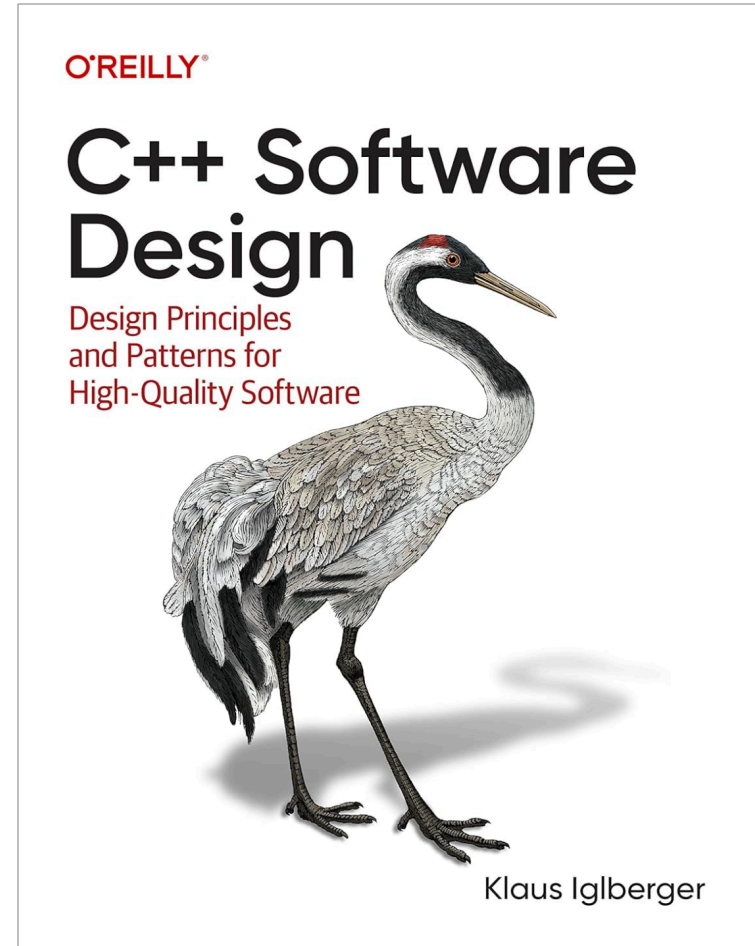
The Observer Pattern and Binary sizes

A few words on the observer pattern

- A behavioural design pattern
- An object can notify other objects about changes in their state



Examples are the courtesy of Klaus Iglberger



The classical interface

```
class AddressObserver : public Observer<Person, Person::StateChange> {
public:
    void update( Person const& person, Person::StateChange property ) override
    { /* manage state change */
};
// ...
AddressObserver addressObserver;

Person homer("Homer", "Simpson");
homer.attach(/* Observer<Person, StateChange> */ &addressObserver);
homer.address("712 Red Bark Lane, Henderson, Clark County, Nevada 89011");
homer.detach(&addressObserver);
```

The modern interface is similar

```
using PersonObserver = Observer<Person, Person::StateChange>;

/* using OnUpdate = std::function<void(Subject const&, StateTag)>; */
PersonObserver addressObserver( /* OnUpdate */
    [/*captured state*/] ( Person const& person, Person::StateChange property ) {
        if( property == Person::forenameChanged ||
            property == Person::surnameChanged )
        { /* ... Respond to changed address */ }
    } );

Person homer("Homer", "Simpson");
homer.attach(&addressObserver);
homer.address("712 Red Bark Lane, Henderson, Clark County, Nevada 89011");
homer.detach(&addressObserver);
```

A surprising result

Version	Binary size	Compile time
classical observer -O0	85.5K	9.04s
modern observer -O0	142K	7.56s
classical observer -O3	35.9K	9.84s
modern observer -O3	39.5K	8.34s
classical observer -Os	47.1K	9.91s
modern observer -Os	40.9K	8.2s

Using function pointers helps

Version	Binary size	Compile time
classical observer -O3	35.9K	9.84s
modern observer with <code>std::function</code> -O3	39.5K	8.34s
modern observer with function pointers -O3	35K	7.48s
modern observer with a function pointer and a lambda -O3	35.1K	7.5s

The modern solution has an advantage but details matter

- Choosing the right design approach is important!
- But when you compare solutions, look at also implementation details
- How you pass functions matters a lot!

Go with templates or function pointers when you can

- Faster
- Cheaper
- But probably not more readable
- Things might change a bit with `std::move_only_function` and with `std::function_ref`

constexpr **functions**

`constexpr` functions

- Might be executed at compile-time
- Guarantees no undefined behaviour
- Guarantees implicit inlining

Let's experiment with 3 versions

- Within one TU
- Within several TUs
- Across libraries

The example is simple

```
// tried also without constexpr and only with inline  
constexpr auto Fun(int v) {  
    return 42 / v;  
}  
  
int main() {  
    const auto f = Fun(6);  
    return f;  
}
```

No big difference for single use

Version	Binary size in bytes
<code>constexpr</code>	16,856
<code>inline</code>	16,856
no modifiers	16,904

- At least not bigger (optimized)
- Intermediary assembler file is never bigger

Still no big difference for multi TU use

Version	Binary size in bytes
constexpr -O0	39,761
non-constexpr -O0	39,681
constexpr -O3	39,425
non-constexpr -O3	39,475

- Utility header + 3 classes using
- Difference is small, below 0.3%
- Changing **Fun** implementation didn't make this bigger

But it matters when it's distributed!

Version	Binary size in bytes
<code>constexpr</code> libutil user -O3	36,778
non- <code>constexpr</code> libutil user -O3	33,370
<code>constexpr</code> all combined -O3	119,329
non- <code>constexpr</code> all combined -O3	185,697

- Utility header distributed in an `.so`
- Its size doesn't change whether its `constexpr` or not
- Libutil users are much smaller with `constexpr`
- Changing `Fun` implementation didn't make this bigger

Compiler and linker flags

Mini-agenda

- The simplest one: `-Os`
- Link time optimization: `-flto`
- `-fdata-sections` / `-ffunction-sections` and `-Wl,--gc-sections`
- `-Wl,--icf=all` or `-Wl,--icf==safe`
- `-Wl,-s` and `-Wl,--strip-all`
- `-Wl,--as-needed`
- `-mllvm -inline-threshold=<n>`
- `-fstack-protector`

The simplest one: `-Os`

- One of the optimization levels
 - `-O[0-3]` for speed
 - `-O[s|z]` for space
- Based on `-O2`
- Skips certain optimizations such as loop unrolling

Link time optimization: `-flto`

- Also referred to as *Inter Procedural Optimization*
- Performed mostly by the linker
- Goes beyond individual object files

Adds an extra step to the compilation process

- Extra steps before compiling to object files
- Translated into intermediary bitcode
- The linker extracts dependency information and optimizes

Different available options

- (Depending on the compiler / linker)
- `-fcto` for default config
- `-fcto=thin` for faster but less aggressive optimization
- `-fcto=full` for maximum optimization

`-fcto=full` / `-fwhole-program` for maximum optimization

- Every TU is treated as a single unit during optimization
- Slows and incurs aggressive inlining
- Might lead to even bigger binary

`-fdata-sections` / `-ffunction-sections` and
`-Wl,--gc-sections`

- Places each global data / function into its own section
- At this point the binary is larger
- `-Wl,--gc-sections` removes the unused sections

Unused sections == unused code?

- The linker includes entire `.text` sections
- Code that you don't use, but others do
- This options removes those
- Don't forget to measure and test!

-Wl,--icf=all or **-Wl,--icf==safe**

- DRY principle applied by the linker
- Identical Code Folding
- Google experiments reported 6% decrease

`-Wl,--icf=all` might be unsafe

- It removes generated code for identical functions
- If functions are keys in associative containers, it might be a problem
- Then you can use `-Wl,--icf=safe`

`-Wl, -s` and `-Wl, --strip-all`

- Removes symbols, debug info and non-essential info for execution
- Useful for production environments and embedded systems
- It makes debugging more challenging
- Measure, because sometimes it only takes but doesn't bring anything

-Wl, --as-needed

- By default all specified libraries are linked
- Excludes unnecessary library dependencies
- LTO might effect this

`-mllvm -inline-threshold=<n>` /

`-finline-limit`

- `n`'s default is 144
- Effects inlining
- Play with it, but measure it

-fstack-protector

- This makes your binary bigger
- Security features to prevent buffer overflow capabilities
- It inserts random values on the stack before the return adress

`-fstack-protector` has several settings

- `-fstack-protector` is the default
- `-fstack-protector-strong` applied to more functions
- `-fstack-protector-all` applied to every function
- `-fstack-protector-explicit` by GCC by using `(__attribute__((stack_protect)))`

**The intersection of ways of coding to
reduce binary size**

Mini-agenda

- RTTI
- Exceptions

Binary sizes and RTTI

What is RTTI?

- *Run-Time Type Information*
- Available for every polymorphic class
- Helps determine the dynamic type of an object
- Turned on by default
- Turn it off with `-fno-rtti` on gcc/clang, `/GR-` on msvc

Gives us `typeid()` and `std::type_info`

- `typeid()` returns `std::type_info`
- might be used to compare the types of two objects
- often queried to get the name of a type (though the name is implementation defined)

What about `dynamic_cast`?

- along the inheritance tree, safely converts pointers and references
 - up
 - down
 - sideways
- we often see it along with loads of `if`s...

Is this good or does it smell?

```
OffRoader* offroader = dynamic_cast<OffRoader*>(car);  
if (offroader) {  
    offroader->turnOnAllWheelDrive();  
}
```

```
Van* van = dynamic_cast<Van*>(car);  
if (van) {  
    van->attachThirdSeatRow();  
}
```

```
Roadster* roadster = dynamic_cast<Roadster*>(car);  
if (roadster) {  
    roadster->removeRoof();  
}
```

Without RTTI it's cleaner!

- We can use a unified interface
- And let the runtime dispatch

What do we gain?

- Clarity!
- Space in terms of LoC written!
- Smaller binary

Smaller, cleaner

Case	Binary size
With <code>dynamic_cast</code> -O0	88.7k
With <code>dynamic_cast</code> -O3	37.3k
Without <code>dynamic_cast</code> , RTTI still on, -O0	88.8k
Without <code>dynamic_cast</code> , RTTI still on, -O3	37.2k
Without <code>dynamic_cast</code> , RTTI off, -O0	88.6k
Without <code>dynamic_cast</code> , RTTI off, -O3	37.0k

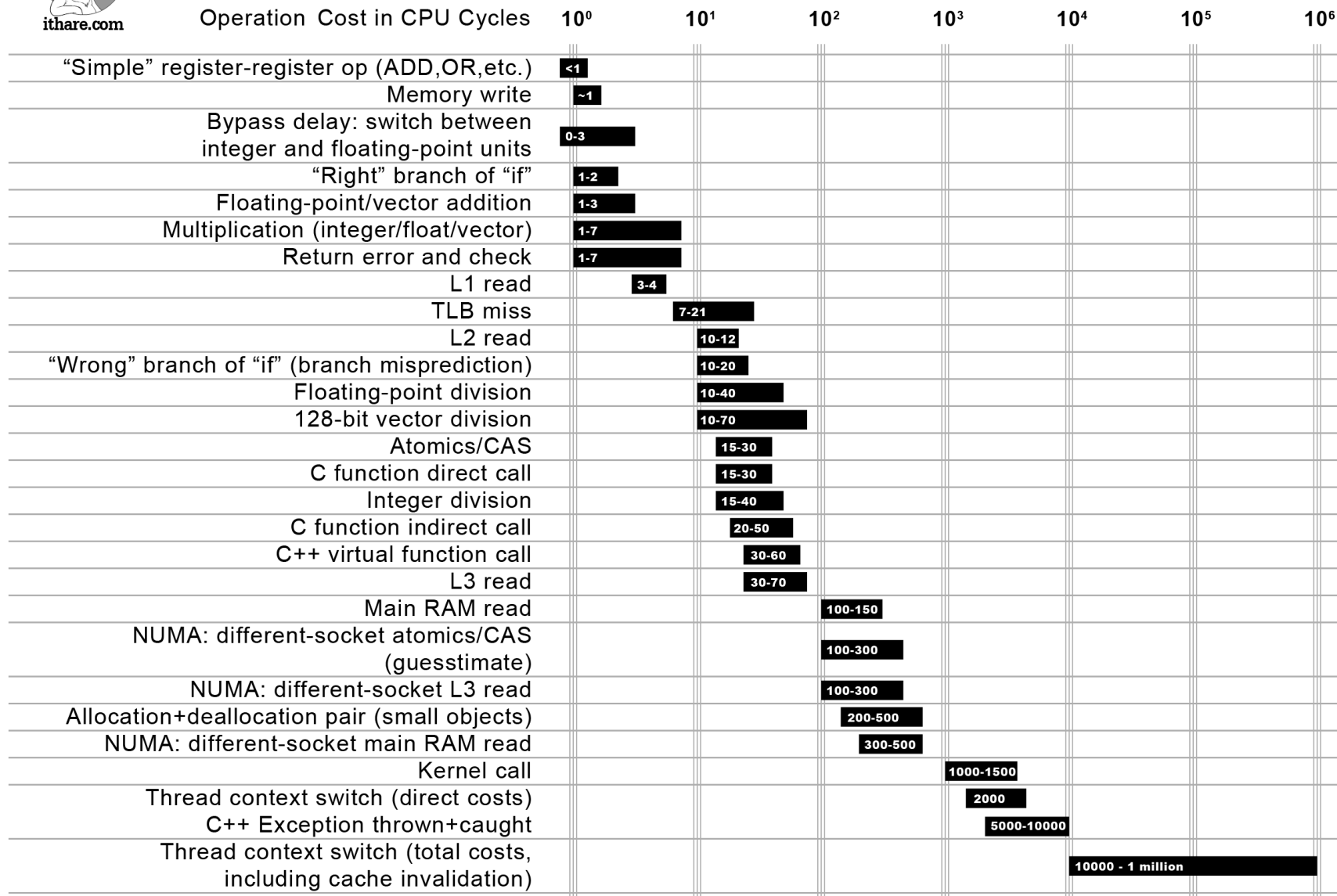
Why would you do that?

- The frequent usage of `dynamic_cast` is bad
- Still we can see it too often...
- So why not banning it when you don't really gain anything?!

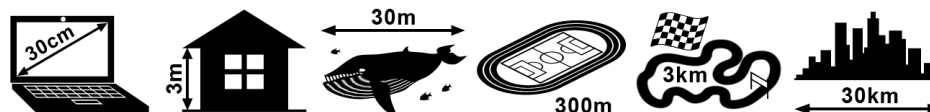
Exceptions and binary sizes



Not all CPU operations are created equal



Distance which light travels while the operation is performed



Are they zero-cost when not thrown?

- Zero run-time costs
- But the compilation time must be slower
- The binary has to store exception tables and unwind information

What if we turn exceptions off?

- Exceptions cannot be thrown
- If they are still thrown, `std::terminate` is called

How does it affect a simple class?

Version		Binary size
default	special functions with exceptions -O0	116,302
default	special functions without exceptions -O0	116,302
default	special functions with exceptions -O3	116,270
default	special functions without exceptions -O3	116,270



What about a more complex case?

A small difference with the observer pattern

Version	Binary size
with exceptions -O0	84,689
without exceptions -O0	83,345
with exceptions -O3	35,137
without exceptions -O3	34,977
with exceptions -Os	36,305
without exceptions -Os	36,017

The difference is not big, but can we spot it?

```
Lfunc_end0:
    .cfi_endproc
    .section      __TEXT,__gcc_except_tab
    .p2align      2
GCC_except_table2:
Lexception0:
    .byte 255          ; @LPStart Encoding = omit
    .byte 255          ; @TType Encoding = omit
    .byte 1            ; Call site Encoding = uleb128
```

```

Lfunc_begin0:
    .cfi_startproc
    .cfi_personality 155, __gxx_personality_v0
    .cfi_lsda 16, Lexception0
; %bb.0:
    stp      x20, x19, [sp, #-32]!           ; 16-byte Folded Spill
    stp      x29, x30, [sp, #16]            ; 16-byte Folded Spill
    add      x29, sp, #16
    sub      sp, sp, #528
    .cfi_def_cfa w29, 16
    .cfi_offset w30, -8
    .cfi_offset w29, -16
    .cfi_offset w19, -24
    .cfi_offset w20, -32

```

Use `noexcept` if you need *some* exceptions!

- specifies whether a function can throw an exception
- `noexcept` and `noexcept(true)` disallows them
- otherwise, `std::terminate`

A bigger binary?

Version	Binary size
modern observer pattern -O0	84,689
modern observer pattern noexcept everywhere -O0	84,785
modern observer pattern -O3	35,137
modern observer pattern noexcept everywhere -O3	35,425

Exception tables everywhere!

```
Lexception0:  
    .byte    255                ; @LPStart Encoding = omit  
    .byte    155                ; @TType Encoding = indirect pcrel sdata4  
    .uleb128 Lttbase0-Lttbaseref0  
// ... it's much longer
```

It's just a bug

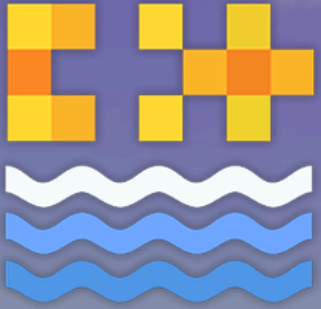
- [an RFC on llvm.org](#)
- So measure first
- Don't just think that it'll be fine!

Main pieces of advice

- Care more about compiler and linker settings
- Prevent inlining if size matters
 - You might not want to default in the header!
- Think twice about exceptions
 - You might need them
 - But make `noexcept` whatever you can
- Don't use virtuals in vain
- Consider turning RTTI off
- Think about how you pass around functions

So what to do with all these?

- Be aware
- You can optimize for many things, including space
- Avoid traps
- Share knowledge



How To Keep Your Binaries Small?

Sandor DARGO

2024